Introduzione all'Intelligenza Artificiale

Incontro 1 — Parte 1: Capire l'Intelligenza Artificiale

Giovanni Della Lunga
giovanni.dellalunga@unibo.it

Prefettura di Milano

20 Ottobre 2026

Prima di cominciare...

Materiale del corso

Tutto il materiale di questo corso è liberamente disponibile al seguente repository GitHub:

`https://github.com/polyhedron-gdl/Seminari-AI`

Nessun prerequisito richiesto

Questo corso non presuppone alcuna competenza tecnica o informatica. Ogni concetto verrà introdotto a partire da zero, con esempi concreti e analogie tratte dall'esperienza comune.

Indice della Parte 1

1. Il framework per l'AI Literacy
2. Che cos'è l'Intelligenza Artificiale?
3. Breve storia dell'Intelligenza Artificiale
4. I due paradigmi dell'IA
5. Il Machine Learning: macchine che imparano
6. Reti neurali e Deep Learning
7. Come impara una rete: la backpropagation
8. Conclusioni della Parte 1

Il framework per l'AI Literacy

Che cos'è l'AI Literacy?

Definizione operativa

L'**AI Literacy** è l'insieme delle competenze che permettono a una persona di comprendere, utilizzare e valutare criticamente i sistemi di intelligenza artificiale.

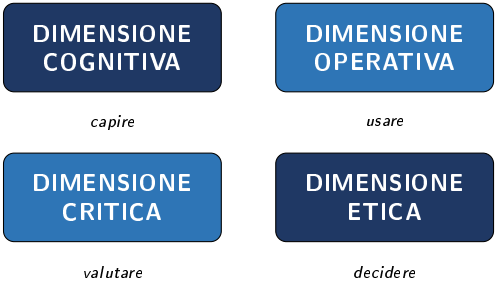
Essere "alfabetizzati" all'IA significa:

- Comprendere **come funziona** l'intelligenza artificiale
- Saper **usare** strumenti basati su IA
- Riflettere sugli **impatti** etici, sociali e istituzionali
- Partecipare attivamente ai **dibattiti** sull'uso dell'IA

Perché serve nella PA?

Perché i sistemi di IA stanno entrando nei procedimenti amministrativi. Chi non li comprende, o li subisce, o li rifiuta a priori: entrambe le posizioni sono inadeguate.

Le quattro dimensioni dell'AI Literacy



Le quattro dimensioni non sono sequenziali: si alimentano a vicenda.

1. Dimensione Cognitiva

Obiettivo

Fornire le basi teoriche e concettuali sull'intelligenza artificiale.

Competenze:

- Comprendere cos'è l'IA e cosa può (e non può) fare
- Conoscere le principali tipologie di IA
  - IA debole / IA forte
  - IA simbolica / IA sub-simbolica
  - Machine Learning, Deep Learning
- Comprendere i principi base del machine learning e delle reti neurali
- Capire il ruolo dei **dati** e i problemi che derivano dai **bias** in essi contenuti

$\Rightarrow$ **È l'oggetto di questa prima parte dell'incontro**

2. Dimensione Operativa

Obiettivo

Abilitare all'uso consapevole e critico di strumenti basati su IA.

Competenze:

- Saper utilizzare strumenti e applicazioni che sfruttano l'IA
- Comprendere e rappresentare dati (*Data Literacy*)
- Saper applicare le funzionalità dell'IA in contesti concreti
- Saper formulare richieste efficaci (*prompting*)

Nel nostro corso

In questa dimensione si collocano la **seconda parte di oggi** (panoramica degli strumenti) e i **laboratori** del secondo incontro.

3. Dimensione Critica

Obiettivo

Sviluppare capacità di valutazione critica delle tecnologie IA e dei loro impatti.

Competenze:

- Analizzare l'affidabilità e la trasparenza degli algoritmi
- Promuovere la **spiegabilità** (*explainability*)
- Comprendere il ruolo del **bias** nei dati e nei sistemi
- Riflettere su limiti e potenzialità dell'IA nei diversi contesti

Dimensione fondamentale per la PA

Quando un sistema automatizzato incide su diritti delle persone, la capacità critica del funzionario non è un optional: è il presidio ultimo della legittimità dell'azione amministrativa.

4. Dimensione Etica

Obiettivo

Comprendere le implicazioni etiche, sociali e normative dell'IA.

Temi centrali:

- **Responsabilità**: chi risponde delle decisioni prese da sistemi automatizzati?
- **Discriminazione**: quali rischi emergono dall'uso di queste tecnologie?
- **Privacy e sicurezza**: impatti sui dati personali
- **Trasparenza**: quale ruolo gioca nella fiducia verso l'IA?
- **Fiducia**: fino a che punto possiamo affidarci alla macchina?

$\Rightarrow$ **Sarà il cuore del terzo incontro**

Che cos'è l'Intelligenza Artificiale?

L'IA nella vita quotidiana

Molto prima di ChatGPT, l'intelligenza artificiale era già nelle nostre tasche. Quasi sempre in modo invisibile.

- Assistenti vocali (Siri, Alexa)
- Raccomandazioni Netflix / Spotify
- Navigatori GPS
- Filtri antispam della posta
- Traduttori automatici
- Riconoscimento facciale del telefono
- Correttore automatico
- ChatGPT, Gemini, Claude...

Definire l'IA non è facile...

- Non esiste una **definizione universalmente accettata**
- L'IA come **simulazione** e **modello** dell'intelligenza umana
- Distinzione classica: **IA debole** vs **IA forte**
- Crescente rilevanza del **Machine Learning** e dei sistemi *data-driven*

Il "paradosso dell'IA" (effetto AI)

Ogni volta che un problema viene risolto dall'informatica, smette di essere considerato "intelligenza artificiale". Ad esempio, il riconoscimento ottico dei caratteri (OCR) era IA negli anni '80. Oggi è semplicemente... uno scanner.

Una definizione di lavoro

Definizione generale

L'**Intelligenza Artificiale** (IA o AI) è la disciplina che studia e sviluppa sistemi informatici capaci di eseguire compiti che normalmente richiederebbero intelligenza umana.

Quali compiti?

- Ragionamento logico
- Apprendimento dall'esperienza
- Riconoscimento di pattern
- Comprensione del linguaggio naturale
- Risoluzione di problemi complessi
- Percezione e interpretazione dell'ambiente

Una definizione "legale"

La definizione dell'AI Act di "Sistema di IA"

Un sistema automatizzato progettato per funzionare con livelli di autonomia variabili e che può presentare adattabilità dopo la diffusione e che, per obiettivi espliciti o impliciti, **deduce dall'input che riceve come generare output** quali previsioni, contenuti, raccomandazioni o decisioni che possono influenzare ambienti fisici o virtuali.

La parola chiave: *deduce*

Un foglio di calcolo con una formula **non** è un sistema di IA: applica una regola che avete scritto voi.
Un sistema che **ricava da sé** la regola dai dati, sì.

IA debole e IA forte

IA debole (*narrow AI*)

- Progettata per un **compito specifico**
- Non ha coscienza né comprensione
- Simula il comportamento intelligente
- **È tutto ciò che esiste oggi**

Esempi: traduttore, riconoscimento facciale, ChatGPT

IA forte (*general AI*)

- Intelligenza **generale** come quella umana
- Coscienza, autoconsapevolezza
- Capace di trasferire conoscenze tra domini
- **Per ora non esiste. È un'ipotesi.**

Esempi: HAL 9000, Skynet, Samantha di *Her*

Punto fondamentale

Tutto ciò di cui parleremo in questo corso è **IA debole**. Potentissima, utilissima, e potenzialmente pericolosa — ma priva di intenzioni, desideri o coscienza.

Demistificare: cosa l'IA non è

- × **Non è cosciente.** Non "vuole" nulla, non ha obiettivi propri, non prova emozioni.
- × **Non capisce** nel senso in cui capiamo noi. Riconosce pattern statistici.
- × **Non è infallibile.** Può sbagliare, uno degli errori più comuni è quello delle cosiddette allucinazioni.
- × **Non è neutrale.** Riflette i dati con cui è stata addestrata, bias compresi.
- × **Non è magia.** È matematica applicata a grandi quantità di dati.

E allora cos'è?

Uno **strumento** straordinariamente potente per riconoscere regolarità, generare testo plausibile e automatizzare compiti ripetitivi. Uno strumento che va conosciuto per essere usato bene.

Breve storia dell'Intelligenza Artificiale

Perché la storia conta

Non è erudizione, è vaccino contro l'hype

L'IA ha attraversato **almeno due cicli** di entusiasmo esagerato seguiti da crolli di fiducia (i cosiddetti *inverni dell'IA*).

Conoscerli aiuta a distinguere ciò che funziona davvero da ciò che viene promesso.



1956: la conferenza di Dartmouth

L'atto di nascita

Nell'estate del 1956, un gruppo di ricercatori si riunisce al Dartmouth College per un seminario estivo. È in quella sede che **John McCarthy** conia il termine "*Artificial Intelligence*".

L'ipotesi di partenza (ambiziosissima):

> *Ogni aspetto dell'apprendimento e ogni caratteristica dell'intelligenza possono in principio essere descritti con tale precisione da poter essere simulati da una macchina.*

- Partecipanti: McCarthy, Minsky, Shannon, Rochester
- Previsione: risolvere il problema in **una generazione**
- Realtà: siamo ancora qui a discuterne, settant'anni dopo

Anni '60: i primi entusiasmi

Successi iniziali che alimentarono l'ottimismo:

- **Logic Theorist** (1956): dimostra teoremi matematici
- **ELIZA** (1966): il primo chatbot, simula uno psicoterapeuta
- **Perceptron** (1958): la prima rete neurale artificiale
- Programmi che giocano a dama e risolvono problemi di algebra

Curiosità: l'effetto ELIZA

ELIZA rispondeva riformulando le frasi dell'utente in forma di domanda. Un meccanismo banalissimo. Eppure molte persone si convincevano di parlare con qualcuno che le capiva davvero.

Lezione ancora attualissima: tendiamo ad attribuire comprensione a un sistema che semplicemente produce testo plausibile.

Anni '70: il primo "inverno dell'IA"

Crisi di fiducia (1974–1980)

- Riduzione drastica dei finanziamenti
- Critiche ai limiti evidenti dei sistemi esistenti
- Il **Report Lighthill** (UK, 1973) demolisce le promesse della ricerca sull'IA

Cosa era andato storto?

- I sistemi funzionavano solo in "*micromondi*" artificiali
- Potenza di calcolo insufficiente
- **Esplosione combinatoria**: i problemi reali hanno troppe possibilità da esplorare
- La traduzione automatica si rivela molto più difficile del previsto

Anni '80: la rinascita con i Sistemi Esperti

L'idea

Non cercare di replicare l'intelligenza generale. Catturare invece la **conoscenza specifica di un esperto umano** in un dominio ristretto, codificandola in regole.

I sistemi più celebri:

- **DENDRAL**: analisi di composti chimici
- **MYCIN**: diagnosi di infezioni batteriche (accuratezza paragonabile a quella dei medici!)
- **R1/XCON**: configurazione di sistemi informatici per DEC (risparmi stimati in decine di milioni di dollari l'anno)

Un mercato da miliardi di dollari. Poi, di nuovo, il crollo.

Il secondo inverno e la rinascita moderna

Secondo inverno (1987–1993)

- I sistemi esperti sono **fragili**: fuori dal loro dominio sbagliano clamorosamente
- Costosissimi da mantenere e aggiornare
- Ogni nuova regola può entrare in conflitto con le precedenti
- Il mercato collassa

La rinascita (dal 2012)

Quattro fattori convergono:

1. **Digitalizzazione**: testi, immagini, suoni e documenti diventano leggibili dalle macchine
2. **Dati**: Internet ne genera e ne fa circolare quantità enormi
3. **Potenza di calcolo**: le GPU rendono possibile addestrare reti profonde
4. **Algoritmi**: nuove tecniche di deep learning

2012: AlexNet vince la competizione ImageNet, dimezzando l'errore.
Da quel momento, tutto accelera.

Dal 2012 a oggi: l'accelerazione

- **2012** — AlexNet: il deep learning conquista la visione artificiale
- **2016** — AlphaGo batte il campione mondiale di Go
- **2017** — Google pubblica *"Attention Is All You Need"*: nasce il **Transformer**
- **2020** — GPT-3: 175 miliardi di parametri
- **2022** — **ChatGPT**: 100 milioni di utenti in due mesi
- **2023–2025** — Claude, Gemini, Copilot: l'IA generativa entra negli uffici

Siamo in un terzo inverno o in una primavera duratura?

Domanda aperta. Ma stavolta c'è una differenza sostanziale rispetto al passato: **i sistemi attuali producono valore economico reale su larga scala**, non solo promesse.

I due paradigmi dell'IA

L'IA non è solo Machine Learning!

IA SIMBOLICA

"Good Old-Fashioned AI"
(GOFAI)

Logica
Regole esplicite

IA SUB-SIMBOLICA

Machine Learning
Deep Learning

Statistica
Apprendimento dai dati

Due modi radicalmente diversi di "fare intelligenza"

IA Simbolica: le regole scritte a mano

Principio

Un esperto umano formalizza la conoscenza in **regole esplicite** del tipo *SE... ALLORA...* Il computer le applica meccanicamente.

Esempio (verifica di un requisito amministrativo):

```
SE (età ≥ 18) E (residenza = "Italia") E (casellario = pulito)
                ALLORA requisito_soddisfatto = VERO
```

Vantaggi

- Completamente **trasparente**
- Ogni decisione è spiegabile
- Non serve nessun dato storico
- Comportamento prevedibile

Limiti

- Rigido: non gestisce l'imprevisto
- Costosissimo da mantenere
- Non gestisce l'incertezza
- Non impara dall'esperienza

IA Sub-simbolica: l'apprendimento dai dati

Principio

Nessuno scrive le regole. Il sistema le **scopre da solo** analizzando grandi quantità di esempi.

Principi base:

- Apprendimento automatico da esempi
- Rappresentazioni distribuite (neuroni, pesi numerici)
- Adattamento statistico
- Riconoscimento di pattern

Categorie principali:

- **Machine Learning classico**: SVM, alberi di decisione, k-means
- **Reti neurali artificiali**: Percettrone, MLP
- **Deep Learning**: CNN, RNN, **Transformer**
- **Apprendimento per rinforzo**: AlphaGo, robotica

Che cosa vuol dire “scopre le regole da solo”?

Richieste di rimborso spese già decise

Importo	Esito
40 €	approvata
85 €	approvata
120 €	approvata
190 €	respinta
260 €	respinta
400 €	respinta

(dati fittizi)

Qual è la regola?

Che cosa vuol dire “scopre le regole da solo”?

Richieste di rimborso spese già decise

Importo	Esito
40 €	approvata
85 €	approvata
120 €	approvata
190 €	respinta
260 €	respinta
400 €	respinta

(dati fittizi)

Qual è la regola?

Probabilmente avete risposto. . .

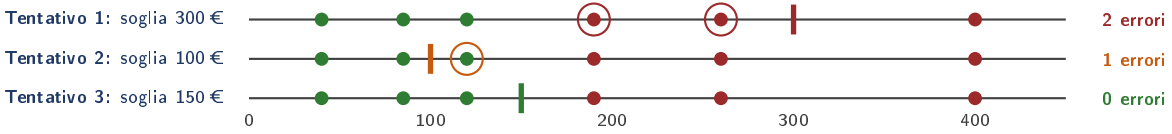
“Si approva fino a circa **150 euro**.”

Ma nessuno vi ha dato quella regola

L’avete **ricavata dagli esempi**.

È esattamente questo che si intende con “la macchina scopre la regola”.

Come lo fa la macchina: prova, conta gli errori, corregge



regola provata: si approva se l'importo è sotto la soglia (|); i cerchi indicano i casi sbagliati

Nessuna intuizione: solo tentativi e correzioni

La macchina propone una regola, **conta quanti casi sbaglia** e la sposta finché gli errori spariscono. "Da solo" non vuol dire "con intelligenza": vuol dire che la regola **non l'ha scritta un programmatore**: l'ha trovata una procedura.

Due avvertenze, che ritroveremo

- **La regola non è unica**: qualunque soglia tra 120 e 190 euro spiega i dati. La macchina ne sceglie una, non "quella vera".
- **Impara ciò che c'è nei dati, non ciò che è giusto**: se le decisioni passate erano incoerenti o distorte, la regola appresa le riproduce.

IA Sub-simbolica: vantaggi e limiti

Vantaggi

- Apprende automaticamente
- Gestisce dati "rumorosi" e incompleti
- Adattabile e flessibile
- Eccelle nel riconoscimento di pattern
- Migliora con più dati

Limiti

- **"Scatola nera"** (*black box*)
- Servono moltissimi dati
- Computazionalmente costoso
- Può "imparare a memoria" (*overfitting*)
- Difficile da correggere

Il problema della spiegabilità

Come sapere *perché* il sistema ha preso una certa decisione?

Domanda decisiva quando quella decisione riguarda una persona e può essere impugnata.

Il futuro: approcci ibridi

La tendenza attuale

Combinare simbolico e sub-simbolico, prendendo il meglio di entrambi.

Esempi di sistemi ibridi:

- **Diagnosi medica**: deep learning per l'analisi delle immagini + regole cliniche per la decisione
- **Veicoli autonomi**: reti neurali per la percezione + pianificazione simbolica per le manovre
- **Assistenti virtuali**: elaborazione neurale del linguaggio + basi di conoscenza strutturate
- **AlphaGo**: reti neurali + ricerca ad albero

Perché interessa alla PA

Un sistema ibrido può essere **potente** dove serve flessibilità e **trasparente** dove serve motivare un provvedimento.

Il Machine Learning: macchine che imparano

Machine Learning: la definizione

Definizione

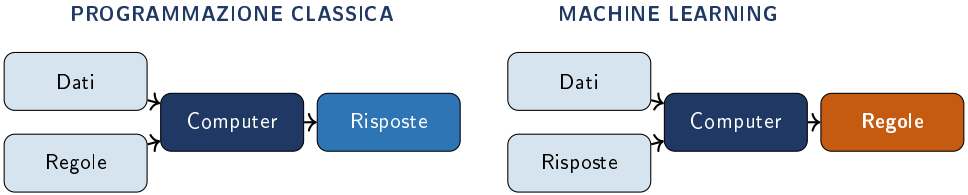
Il **Machine Learning** è un approccio in cui l'algoritmo **scopre automaticamente le regole** analizzando grandi quantità di dati, senza che queste regole vengano programmate esplicitamente.

Caratteristiche:

- Le regole **emergono** dall'analisi dei dati
- Il comportamento **varia** in base ai dati di addestramento
- L'algoritmo può gestire situazioni **non previste** dal programmatore
- La logica interna è spesso **complessa e non direttamente interpretabile**

Ognuna di queste quattro caratteristiche è insieme un pregio e un rischio.

Programmazione classica vs Machine Learning



L'inversione concettuale

Nel ML non diciamo più alla macchina *come* fare qualcosa. Le mostriamo **migliaia di esempi** di cose fatte bene, e lei ricava da sola il metodo.

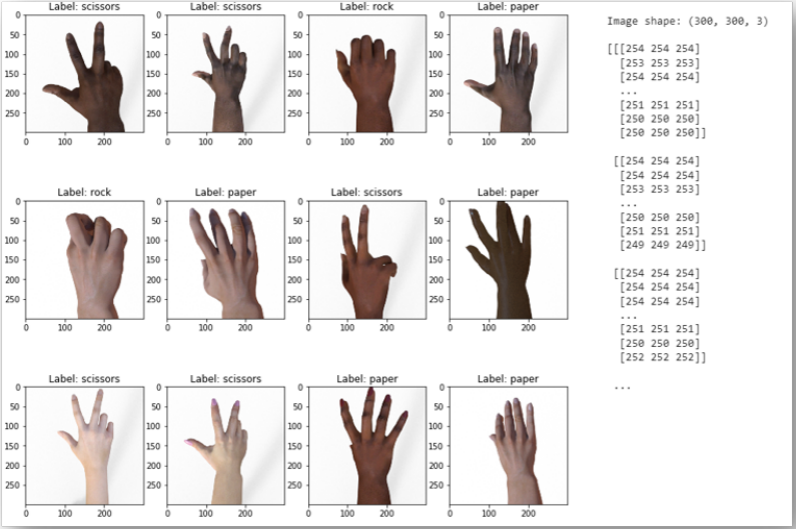
Un esempio concreto: il riconoscimento di immagini

Un problema semplice (si fa per dire!)



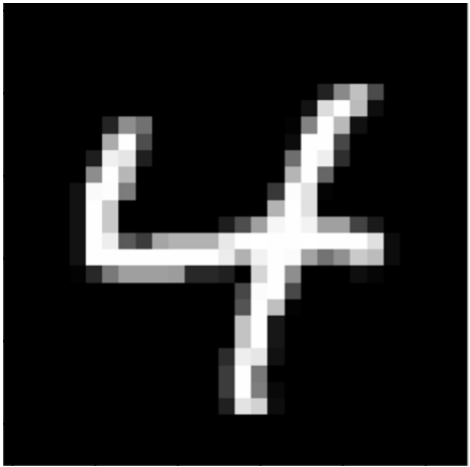


Un esempio concreto: il riconoscimento di immagini



Un esempio concreto: il riconoscimento di immagini

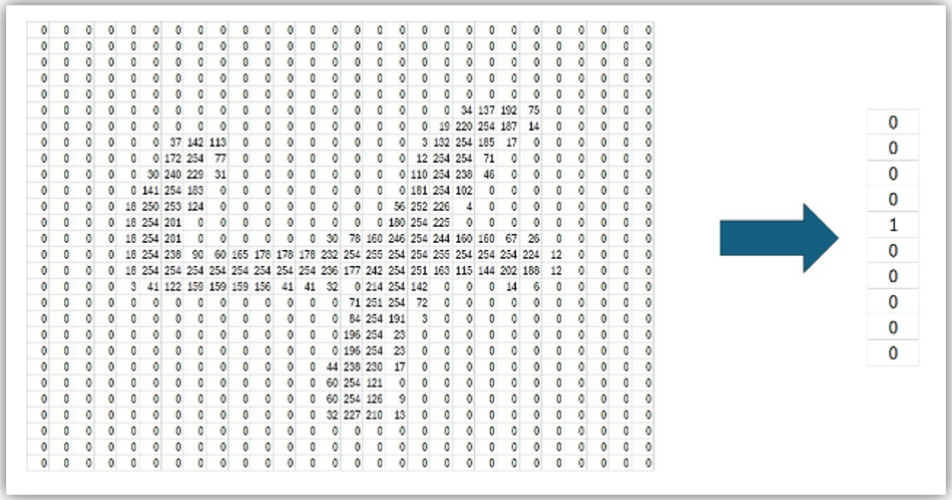
Cosa "vede" una macchina



0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	34	137	192	75	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	15	220	254	187	14	0	0	0	0
0	0	0	0	0	0	37	142	113	0	0	0	0	0	0	0	0	0	3	132	254	185	17	0	0	0	0	0
0	0	0	0	0	0	172	254	77	0	0	0	0	0	0	0	0	0	12	254	254	71	0	0	0	0	0	0
0	0	0	0	0	90	240	220	31	0	0	0	0	0	0	0	0	0	110	254	238	46	0	0	0	0	0	0
0	0	0	0	0	141	254	183	0	0	0	0	0	0	0	0	0	0	181	254	102	0	0	0	0	0	0	0
0	0	0	0	18	250	253	124	0	0	0	0	0	0	0	0	0	56	252	226	4	0	0	0	0	0	0	0
0	0	0	0	18	254	201	0	0	0	0	0	0	0	0	0	0	180	254	225	0	0	0	0	0	0	0	0
0	0	0	0	18	254	201	0	0	0	0	0	0	0	30	78	160	246	254	244	160	160	67	26	0	0	0	0
0	0	0	0	18	254	238	90	60	165	178	178	178	232	254	205	254	254	205	254	254	254	224	12	0	0	0	0
0	0	0	0	18	254	254	254	254	254	254	254	254	206	177	242	254	251	163	115	144	202	188	12	0	0	0	0
0	0	0	0	3	41	122	159	159	159	156	41	41	32	0	214	254	142	0	0	0	14	6	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	71	251	254	72	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	84	254	191	3	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	196	254	23	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	196	254	23	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	44	239	230	17	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	60	254	121	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	60	254	126	9	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	32	227	210	13	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Un esempio concreto: il riconoscimento di immagini

Riconoscere vuol dire Classificare


0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Un esempio concreto: il rilevamento delle frodi

Come lo faremmo con le regole (approccio simbolico)

`SE (importo > 5000) E (paese ≠ paese_abituale) ALLORA sospetta`

. . . e altre 400 regole, tutte scritte a mano, tutte da aggiornare.

Come lo fa il Machine Learning

1. L'algoritmo analizza **milioni di transazioni passate**, già etichettate come "fraudolenta" o "legittima"
2. Identifica **automaticamente** i pattern sospetti
3. Impara a distinguere le transazioni normali da quelle fraudolente
4. **Non esiste alcuna regola esplicita scritta da un programmatore**

Funziona molto meglio. Ma se la banca vi blocca la carta, non è sempre semplice dirvi esattamente perché.

I tre tipi di apprendimento

1. Apprendimento supervisionato

Il sistema riceve esempi **con la risposta corretta**.
Es.: 10.000 email già etichettate come spam / non spam.
$\Rightarrow$ È il caso sul quale ci soffermeremo maggiormente.

2. Apprendimento non supervisionato

Il sistema riceve dati **senza etichette** e deve trovare da solo delle strutture.
Es.: raggruppare i cittadini in categorie simili per tipologia di richiesta.

3. Apprendimento per rinforzo

Il sistema impara per **tentativi ed errori**, ricevendo "premi" quando fa bene.
Es.: AlphaGo, che ha imparato giocando milioni di partite contro se stesso.

I dati: la materia prima dell'IA

Principio fondamentale

Un sistema di IA non può essere migliore dei dati con cui è stato addestrato.

Cosa serve a un dataset per essere buono:

- **Quantità**: migliaia, spesso milioni di esempi
- **Qualità**: dati corretti, puliti, ben etichettati
- **Rappresentatività**: devono rispecchiare la realtà su cui il sistema lavorerà
- **Diversità**: devono coprire tutti i casi rilevanti, non solo quelli frequenti

Dove nasce il bias

Se i dati storici riflettono una discriminazione passata, il sistema la imparerà e la riprodurrà — con l'aggravante di farlo su scala industriale e con l'aura di oggettività del calcolo.

Ne parleremo diffusamente nel secondo incontro.

Riconoscere vuol dire classificare

Per una macchina, tutto è numeri

Un'immagine è una griglia di numeri (l'intensità di ogni pixel). Un testo è una sequenza di numeri. Un cittadino, in una pratica, è una lista di numeri (età, reddito, anzianità...).

Il compito più comune: classificare

Dato questo insieme di numeri in ingresso, **a quale categoria appartiene**?

- Questa immagine è un "2" o un "7"?
- Questa transazione è "legittima" o "fraudolenta"?
- Questa istanza è "completa" o "incompleta"?

Riconoscere è **classificare**. Ma a volte la domanda non è "*di che tipo è?*" bensì "*quanto vale?*"

Due domande diverse: *di che tipo?* e *quanto?*

Classificare

La risposta è una **categoria**

- La pratica è *completa* o *incompleta*?
- La mail è *spam* o no?
- La transazione è *sospetta*?

Stimare un numero (*regressione*)

La risposta è un **valore**

- Quanti *giorni* servono per evadere questa pratica?
- Quante *istanze* arriveranno il mese prossimo?
- Qual è il *valore* di questo immobile?

Lo stesso modo di imparare

In entrambi i casi la macchina impara da **esempi con la risposta giusta**: prova, misura l'errore, corregge. Cambia soltanto il **tipo** di risposta che deve dare.

E, come vedremo, entrambe le domande sono problemi **geometrici**.

I dati come punti in uno spazio

Per un algoritmo, un dato non è una riga di tabella: è un punto in uno spazio geometrico

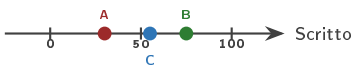
I dati (una tabella)

Candidato	Scritto	Orale
A	30	40
B	75	80
C	55	50

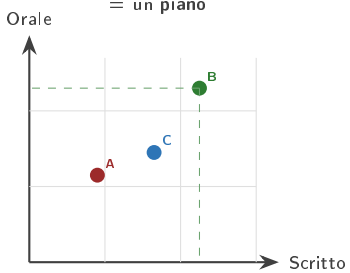
*Ogni **riga** è un candidato.*
*Ogni **colonna** è una caratteristica.*

diventa

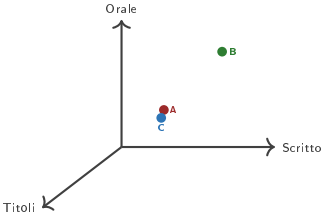
1 caratteristica
= una **retta**



2 caratteristiche
= un **piano**



. . . e oltre le tre dimensioni



3 caratteristiche = uno **spazio**

N caratteristiche = spazio a N dimensioni

Non possiamo più **disegnarlo**. . .
. . . ma la **matematica funziona allo stesso modo**.

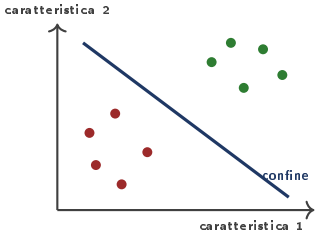
100 caratteristiche $\Rightarrow$ spazio a 100 dimensioni.

Il messaggio da portare a casa

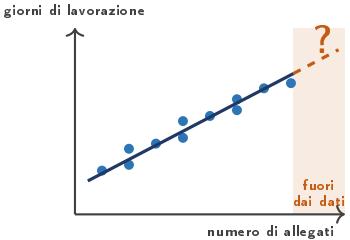
Per un algoritmo, un dato è sempre un **punto** in uno spazio. Tutto ciò che fa — misurare **distanze**, tracciare **confini**, formare **gruppi** — è **geometria**.

Punti vicini = casi simili. *Un confine nello spazio = una regola di decisione.*

Due compiti, due geometrie



Classificare = tracciare un **confine**
da che parte cade il nuovo caso?



Stimare = tracciare una **tendenza**
quale valore corrisponde al nuovo caso?

Un'avvertenza sulla stima

Una stima è affidabile **dentro** l'intervallo dei dati già visti. In una situazione mai vista (ad esempio un afflusso di istanze dovuto a un'emergenza) la macchina **prolunga la tendenza** e può sbagliare con totale sicurezza.

(dati fittizi) E se gli esempi non hanno nessuna etichetta? Si possono solo **raggruppare**: è il prossimo esempio.

Un esempio pratico: la segmentazione della clientela

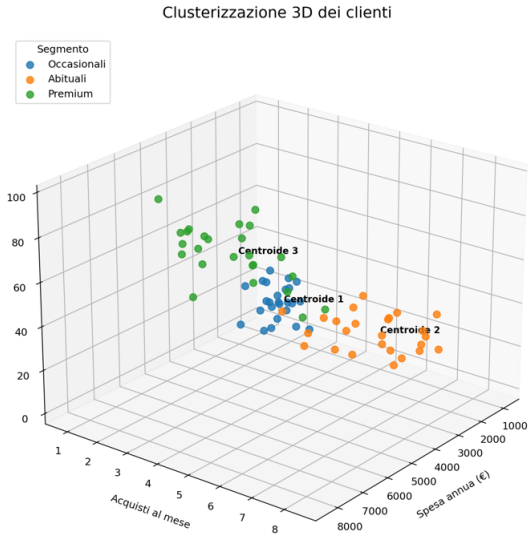
Apprendimento *non supervisionato*

Nessuna etichetta: l'algoritmo riceve solo i dati e **trova da sé i gruppi**.

- Ogni punto è un **cliente**, descritto da tre caratteristiche: spesa annua, acquisti al mese, quota premium
- Comportamenti simili $\Rightarrow$ **punti vicini**: emergono **tre gruppi** (*cluster*)
- I **centroidi** indicano il profilo medio di ciascun gruppo

Da notare

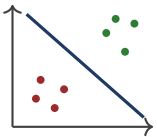
L'algoritmo trova i gruppi, ma **i nomi li diamo noi**: "occasionali", "abituali", "premium" sono una nostra interpretazione.



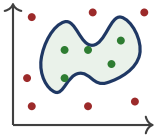
Chi traccia il confine?

La domanda che resta aperta

Classificare, stimare, raggruppare: è sempre **tracciare una forma** nello spazio dei dati. Ma **chi trova la forma giusta**?



nei nostri esempi:
una retta



nei casi reali:
una forma complicata

Molti strumenti possibili

- **Regressione lineare**: tendenze rettilinee
- **Alberi di decisione**: confini "a gradini"
- **Algoritmi di raggruppamento**: gruppi (come la segmentazione)

Ognuno sa tracciare forme di un certo tipo.

Perché le reti neurali hanno vinto

Costruiscono **da sé** forme di qualunque complessità, anche in migliaia di dimensioni, **combinando tanti pezzi semplici**: i **neuroni**.

Più potenti, non sempre più adatte

Con dati tabellari (pratiche, importi, scadenze) i metodi semplici spesso bastano, e hanno un vantaggio decisivo: **si possono spiegare**.

Non solo separare i dati: rappresentarli meglio

Il problema della rappresentazione

Abbiamo visto che ogni dato può essere rappresentato come un punto in uno spazio geometrico.
Ma non tutte le rappresentazioni sono ugualmente utili: un problema difficile in uno spazio può diventare semplice in un altro.

Un esempio: due gruppi concentrici

Immaginiamo due gruppi di punti disposti ad anello, uno all'interno dell'altro.

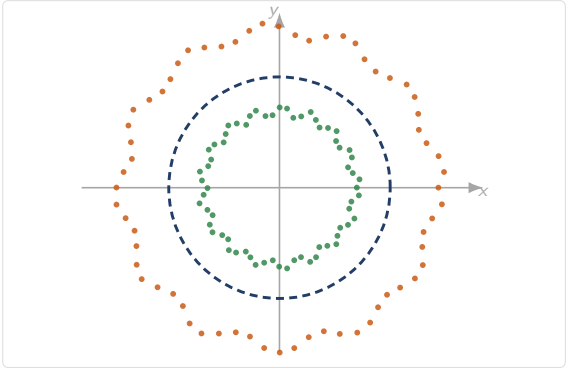
- Nella rappresentazione originale, nessuna retta può separarli.
- Cambiando la rappresentazione, i due gruppi possono diventare separabili con una semplice retta.

L'idea fondamentale

Il Machine Learning può imparare non soltanto come distinguere i dati, ma anche come rappresentarli per rendere più facile il compito.
Questo è uno dei principi fondamentali del Deep Learning: imparare rappresentazioni progressivamente più utili.

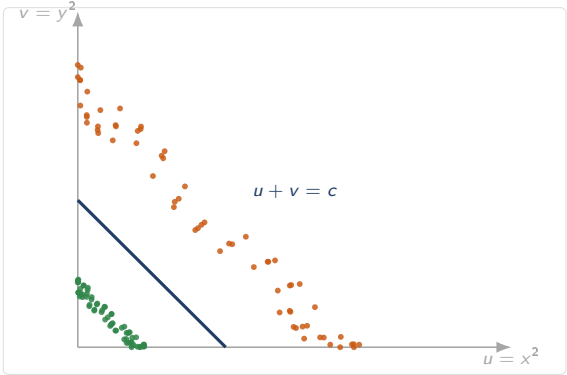
Non solo separare i dati: rappresentarli meglio

Coordinate originali: (x, y)



Serve un confine curvo

Nuove caratteristiche: $(u, v) = (x^2, y^2)$



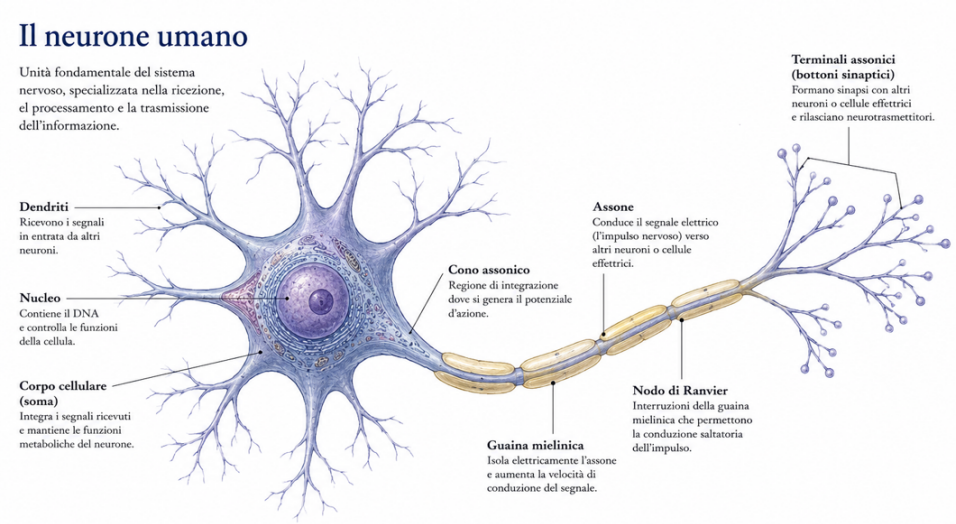
Adesso basta una retta!

L'idea fondamentale

Il Machine Learning può **imparare nuove rappresentazioni**, non soltanto separare i dati. Qui scegliamo noi la trasformazione; nel Deep Learning può essere appresa dai dati.

Reti neurali e Deep Learning

Il Neurone: L'ispirazione biologica



L'idea rivoluzionaria

Ispirazione biologica

Il cervello umano elabora informazioni attraverso miliardi di **neuroni interconnessi**. Ogni neurone riceve segnali, li pesa, e decide se "attivarsi".

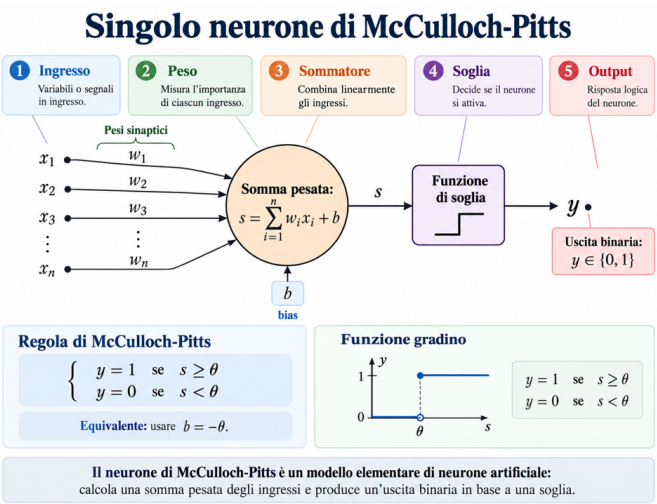
Il neurone artificiale

$$\text{output} = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

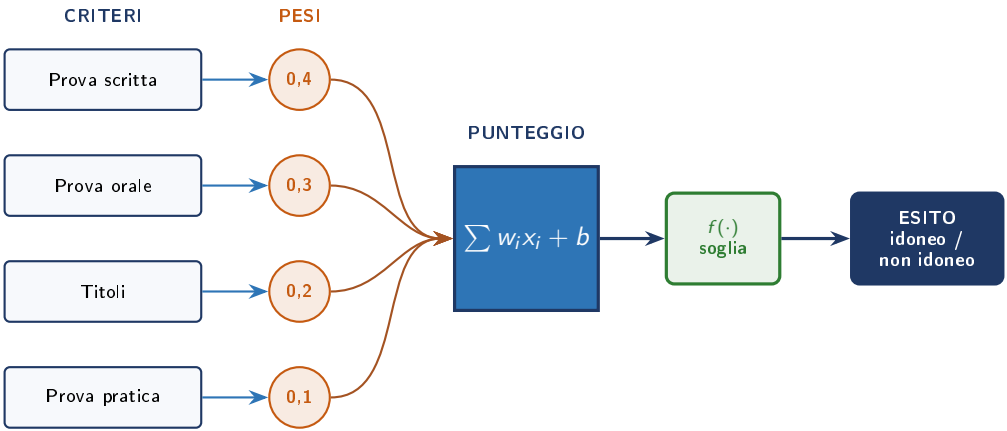
dove:

- x_i sono gli **input** (le informazioni in ingresso)
- w_i sono i **pesi** (quanto conta ciascun input)
- b è la **soglia** (*bias*)
- f è una **funzione di attivazione** non lineare

Il Neurone: Il Modello Matematico



Il neurone come “decisore”: il commissario di concorso



$$\text{Punteggio} = f\left(\text{Scritto} \times 0{,}4 + \text{Orale} \times 0{,}3 + \text{Titoli} \times 0{,}2 + \text{Pratica} \times 0{,}1 + b\right)$$

Il neurone come “decisore”: la differenza cruciale

Dove finisce l’analogia

- Nel bando di concorso i pesi sono **scelti e pubblici**: li fissa un regolamento, e chiunque può leggerli
- Nel neurone artificiale **non li ha scelti nessuno**: sono stati **ricavati dai dati**, analizzando moltissimi casi già decisi

Stessa struttura, trasparenza opposta.
Un bando si può leggere e contestare.
I pesi di una rete, spesso, no.

Le funzioni di attivazione: la decisione!

La regola del concorso

Il candidato riceve un **punteggio complessivo** z (la somma pesata dei criteri).

Il bando fissa la regola:

se $z \geq 60 \Rightarrow$ **IDONEO**

se $z < 60 \Rightarrow$ **NON IDONEO**

Tre candidati:

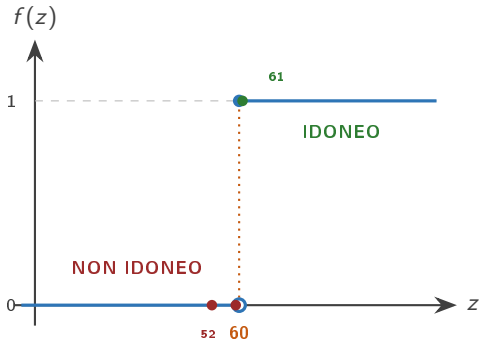
$z = 52$	$z = 59$	$z = 61$
0	0	1

*Fra 59 e 61 il punteggio cambia di **poco**...*
*...ma l'esito cambia **tutto**.*

stessa cosa

$\equiv$

La funzione a gradino (*step*)



$$f(z) = \begin{cases} 1 & \text{se } z \geq 60 \\ 0 & \text{se } z < 60 \end{cases}$$

Le funzioni di attivazione: la decisione!

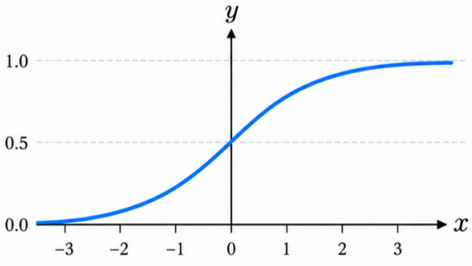
Che cosa fa la funzione di attivazione

- Trasforma un valore continuo (il punteggio z) in una **decisione**.
- Il gradino è la sua forma più semplice: tutto o niente.
- **Nelle reti neurali** si usano gradini "ammorbiditi" per avere sfumature invece di un salto netto — ma l'idea di fondo è questa.

Le funzioni di attivazione: la decisione!

Sigmoide

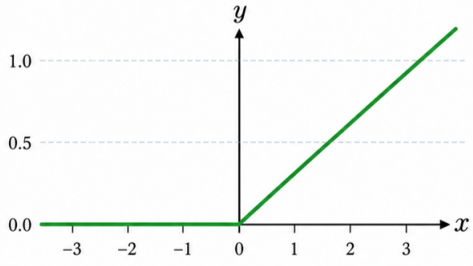
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Sigmoide: comprime l'input nell'intervallo $(0, 1)$; utile per interpretazioni probabilistiche, ma può soffrire di saturazione.

ReLU

$$f(x) = \max(0, x)$$

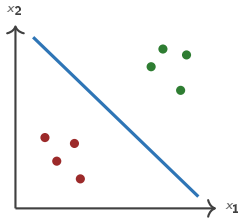


ReLU: vale 0 per input negativi e cresce linearmente per input positivi; è semplice, efficiente e molto usata nelle reti profonde.

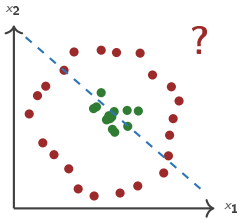
Ma un solo neurone che cosa sa fare?

Un neurone traccia *una* linea

Nello spazio delle caratteristiche, un singolo neurone separa i punti con **una sola retta**: "di qua sì, di là no".



Qui una retta basta

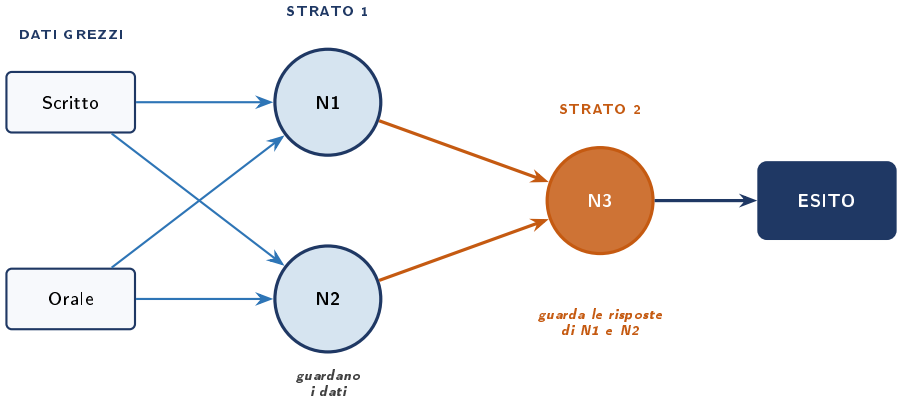


Qui nessuna retta basta

Quando una retta non basta

Per isolare il gruppo centrale servono **più linee**, combinate tra loro — cioè **più neuroni**. È da qui che nasce l'idea di metterli **a strati**.

Più neuroni, a strati: scomporre e ricomporre



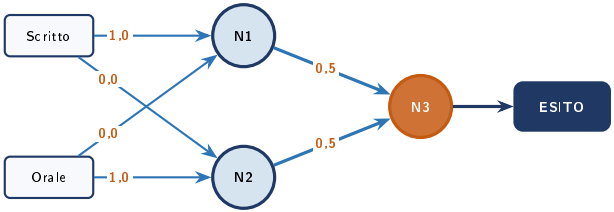
Lo strato 1 *scompone*

N1 e N2 guardano i dati grezzi: una domanda semplice ciascuno, **una retta** a testa.

Lo strato 2 *ricompone*

N3 non vede i dati. Vede le **risposte** di N1 e N2: coglie una relazione che nessuno dei due, da solo, coglierebbe.

Ogni freccia è un numero: i pesi



Che cosa dicono i numeri

- N1 riceve **1,0** dallo scritto e **0,0** dall'orale: **guarda solo lo scritto**
- N2, al contrario, **guarda solo l'orale**
- N3 dà **lo stesso peso** (0,5) alle due risposte

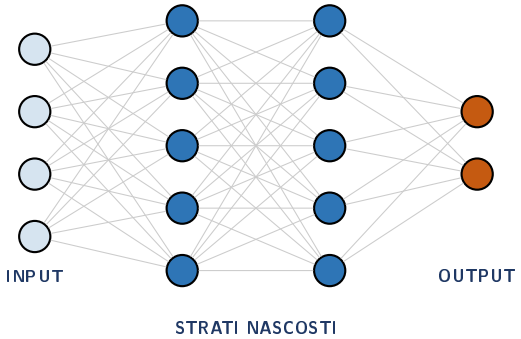
Peso **zero** = "non conta"; peso **alto** = "conta molto".

Contiamo i numeri da regolare

Questa rete (2–2–1):	6 pesi + 3 soglie = **9 numeri**
La rete della prossima slide (4–5–5–2):	55 pesi + 12 soglie = **67 numeri**
GPT-3 (2020):	**175 miliardi** di numeri

In una rete reale i numeri non sono mai così "puliti": qui sono scelti per essere leggibili.

Dalla singola unità alla rete profonda



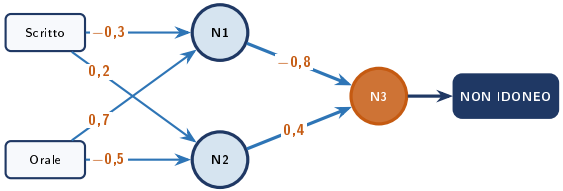
Deep Learning

"**Profondo**" significa semplicemente: *con molti strati nascosti*. I modelli moderni ne hanno decine, a volte centinaia. Ogni strato impara caratteristiche via via più astratte.

Stessa rete, numeri diversi

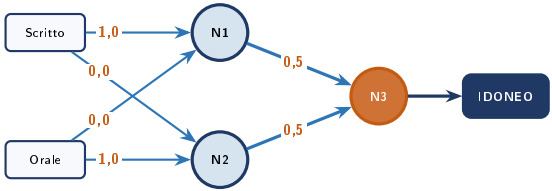
Un candidato: scritto 80, orale 75 $\Rightarrow$ risposta giusta: idoneo

Prima: pesi casuali



risposta sbagliata

Dopo: pesi appresi

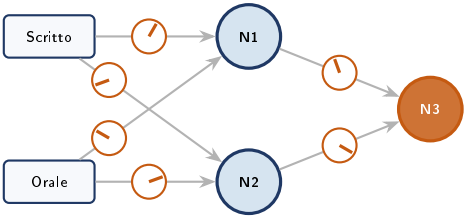


risposta giusta

La forma è identica: cambiano solo i numeri

Una rete addestrata e una non addestrata hanno **la stessa architettura**. Tutto ciò che la rete "sa" **sta nei pesi**. Imparare significa **trovare i numeri giusti**: come, lo vediamo adesso.

I pesi come manopole



Una rete è un mixer

Ogni connessione ha la sua **manopola**: quanto la si gira decide quanto quel segnale conta. **Imparare** = girare le manopole, un pochino alla volta, finché il risultato è giusto.

Dove l'immagine si incrina

Un fonico gira poche manopole **sapendo che cosa vuole ottenere**. La rete ne gira **milioni insieme**, guidata solo dalla misura dell'errore, senza "sapere" che cosa rappresenti ciascuna.

Che cos'è, materialmente, un "modello"

Il nostro modello, per intero

```
1.0   0.0   0.0   1.0
      0.5   0.5
-60   -60   -0.75
```

6 pesi e 3 soglie: **questo è tutto**

Un modello vero

Lo stesso, in scala: un **file di numeri**. GPT-3, ad esempio, sono 175 miliardi di numeri: circa **350 gigabyte**.

Per usarlo serve anche un programma che esegua i calcoli, ma la "conoscenza" sta tutta lì.

Perché interessa a un'amministrazione

- Quei numeri **non si possono leggere**: da qui il problema della **spiegabilità**
- Contengono ciò che è stato appreso dai dati, **bias compresi**
- Spesso restano **presso il fornitore**: l'ente usa il servizio, non possiede i pesi. Esistono anche modelli "a pesi aperti", scaricabili e installabili

Come impara una rete: la backpropagation

Abbiamo una rete... ma come impara?

Dove siamo arrivati

Sappiamo com'è **fatta** una rete: neuroni organizzati in strati, ognuno con i suoi **pesi**. Sappiamo che i pesi determinano il comportamento della rete.

La domanda che abbiamo lasciato in sospeso

All'inizio i pesi sono **casuali**: la rete non sa fare nulla, sbaglia tutto.

Come fa a passare da "*tutti i pesi a caso*" a "*i pesi giusti*"?

Chi le dice quali pesi cambiare, e di quanto?

È il problema dell'**apprendimento**. E la sua soluzione è ciò che ha reso possibile tutto il deep learning moderno.

La dimensione del problema

Non è una questione da poco

Una rete moderna può avere svariati **miliardi**di pesi. Ognuno va regolato al valore giusto.

Perché non basta "provare a caso"

- Provare tutte le combinazioni possibili di pesi è **impossibile**: sono di più degli atomi dell'universo
- Cambiare un peso a caso e vedere se va meglio richiederebbe **tempi geologici**
- Serve un metodo che dica, per **ogni** peso, **in che direzione** spostarlo per ridurre l'errore

Questo metodo esiste, e ha un nome:
backpropagation **— la "propagazione all'indietro" dell'errore.**

L'apprendimento in due movimenti

Ogni "lezione" della rete ha due fasi

La rete impara guardando **un esempio alla volta**, di cui conosce già la risposta giusta. Per ogni esempio compie due movimenti:

1. In avanti (*forward*)

L'esempio entra dagli **input**, attraversa gli strati, ed esce una **risposta**.

$\longrightarrow\longrightarrow\longrightarrow$

"Ecco cosa penso io."

2. All'indietro (*backward*)

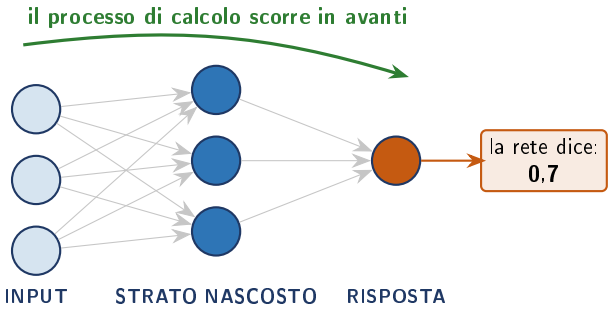
Si confronta la risposta con quella giusta, si misura l'**errore**, e lo si **ripercorre a ritroso**.

$\longleftarrow\longleftarrow\longleftarrow$

"Ecco di chi è la colpa, e di quanto."

La prima fase produce una risposta.
La seconda distribuisce la responsabilità dell'errore.

La fase in avanti: la rete “dice la sua”



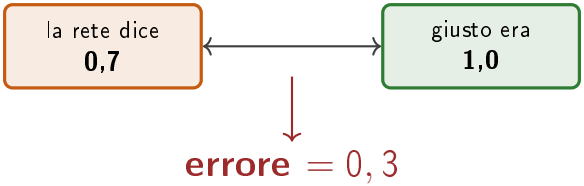
Che cosa succede

Ogni neurone riceve i segnali dallo strato precedente, li combina con i **suoi pesi attuali**, applica la sua funzione di attivazione, e passa il risultato allo strato dopo. Alla fine esce **un numero**: la risposta della rete.

Misurare l'errore

Il confronto con la verità

Per ogni esempio conosciamo la risposta **giusta**. Basta confrontarla con quella **data dalla rete**.



L'obiettivo di tutto il processo

Rendere questo errore il più piccolo possibile, su *tutti* gli esempi.

Se l'errore è zero, la rete risponde sempre come dovrebbe: ha imparato.

L'intuizione chiave: risalire la catena dei pesi

Un'analogia dal mondo delle organizzazioni

Immaginate una decisione sbagliata che esce da un ufficio complesso, frutto del lavoro di molte persone su più livelli.

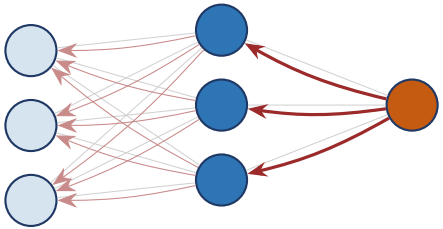
Volete **correggere il processo**. Che cosa fate?

Non modifichiamo i dati a caso: risaliamo la catena dei contributi

- Partite dal **risultato sbagliato** (l'output)
- Chiedete a chi ha prodotto la decisione finale: "*su quali contributi ti sei basato?*"
- Risalite a chi ha fornito quei contributi, e così via **all'indietro**
- A ogni livello capite **chi ha pesato di più** sull'errore

La *backpropagation* fa esattamente questo:
distribuisce la "responsabilità" dell'errore all'indietro, neurone per neurone.

La propagazione all'indietro



L'errore viaggia al contrario

L'errore finale viene **ridistribuito** verso gli strati precedenti. Ogni neurone riceve una **quota di responsabilità** proporzionale a quanto ha contribuito al risultato sbagliato.

Chi ha pesato di più, si corregge di più

Il principio di equità dell'apprendimento

Non tutti i pesi sono responsabili allo stesso modo. La *backpropagation* stabilisce, per **ciascun peso**, quanto ha contribuito all'errore.

Peso molto responsabile

- Ha spinto forte nella direzione sbagliata
- Riceve una **correzione grande**

Peso poco responsabile

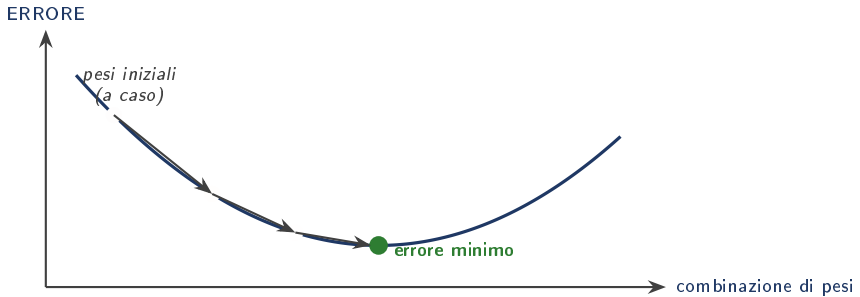
- Ha contribuito poco all'errore
- Riceve una **correzione piccola**

Ogni peso viene spostato di un passettino nella direzione che avrebbe reso l'errore un po' più piccolo.

Scendere la collina dell'errore

Un'immagine per il "passettino"

Immaginate l'errore come l'**altitudine** di un paesaggio di colline. Ogni combinazione di pesi è un punto su quel paesaggio. Vogliamo arrivare **in fondo alla valle**: l'errore minimo.



Ogni correzione è **un passo in discesa**. La *backpropagation* ci dice **da che parte è la discesa**.

Perché un passo *piccolo*?

La velocità dell'apprendimento (*learning rate*)

La dimensione del passo è una scelta delicata.

Passi troppo grandi

- Si rischia di **scavalcare** la valle
- L'errore rimbalza da un lato all'altro
- L'apprendimento **non si stabilizza**

Passi troppo piccoli

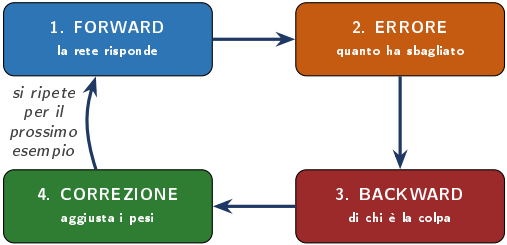
- Si scende, ma **lentissimamente**
- Servono troppe ripetizioni
- Si rischia di fermarsi in una **conca secondaria**

Il giusto compromesso

Un passo **né troppo grande né troppo piccolo** è uno dei parametri che chi costruisce la rete deve regolare con cura.

È più un'arte artigianale che una scienza esatta.

Il ciclo completo dell'apprendimento



Ripetere, ripetere, ripetere

Questo ciclo si ripete **per ogni esempio**, e poi da capo per tutti gli esempi, **molte volte** (ogni giro completo si chiama *epoca*). A ogni giro l'errore cala un pochino.

Milioni di esempi $\times$ molte epoche $=$ miliardi di piccole correzioni.

Che cosa "impara" davvero la rete

Alla fine del processo

Dopo miliardi di correzioni, i pesi si sono assestati su valori che fanno dare alla rete risposte **quasi sempre giuste** sugli esempi visti.

Ma attenzione a non attribuire troppo

- La rete **non ha capito** nulla nel senso umano
- Ha solo trovato una **combinazione di numeri** che minimizza l'errore
- "Imparare", qui, significa **regolare pesi**, non comprendere
- È lo stesso motivo per cui poi la rete può **allucinare** con sicurezza: non sa *perché* risponde così

Ritroviamo qui, dal di dentro, i limiti che avevamo enunciato all'inizio: potenza senza comprensione.

In tre frasi

1. La rete parte con pesi **a caso** e sbaglia. Misuriamo di **quanto** sbaglia.
2. L'errore viene **propagato all'indietro** (*backpropagation*): ogni peso scopre la **sua quota di responsabilità**.
3. Ogni peso si sposta di un **passettino** nella direzione che riduce l'errore. Si ripete milioni di volte.

Imparare, per una rete neurale, significa
correggere milioni di manopole un pochino alla volta,
finché l'errore non diventa piccolo.

Una curiosità storica

Perché questo metodo è così importante

L'algoritmo di *backpropagation* è stato reso celebre a metà degli anni '80. Prima di allora, non si sapeva come addestrare reti con molti strati in modo efficiente: fu uno dei motivi del "*secondo inverno dell'IA*".

Il paradosso

L'idea di fondo è **matematicamente semplice** (è la regola per calcolare come una piccola variazione a monte influenza il risultato a valle).

Eppure senza di essa **niente** di ciò che chiamiamo oggi intelligenza artificiale — ChatGPT, Claude, il riconoscimento di immagini — sarebbe possibile.

Una piccola idea di calcolo, moltiplicata per miliardi di pesi, ha cambiato il mondo.

Cosa serve per addestrare una rete profonda

Dati

Milioni di esempi

Più è grande il modello, più dati richiede

Calcolo

Migliaia di GPU

addestrare un modello di frontiera richiede investimenti alla portata di pochissime aziende al mondo

Algoritmi

Tecniche raffinate

Backpropagation, ottimizzatori, regolarizzazione

Una conseguenza importante per la PA

Addestrare da zero un grande modello è alla portata di pochissimi soggetti al mondo.
La PA sarà quasi sempre **utilizzatrice** di modelli sviluppati da altri — il che sposta il problema dalla *costruzione* alla **valutazione** e alla **governance** dei sistemi.

Conclusioni della Parte 1

Punti chiave da portare a casa

Sui fondamenti

- L'IA ha radici antiche ma uno sviluppo recente e accelerato
- **Non è solo machine learning**: esistono approcci diversi, con vantaggi e limiti complementari
- Tutto ciò che esiste oggi è **IA debole**: potente ma priva di comprensione e di intenzioni

Sui sistemi che useremo

- Il ML **impara dai dati**: la qualità dei dati determina la qualità del sistema
- Le reti profonde sono potenti ma sono **scatole nere**

L'IA è uno strumento potente che richiede comprensione, uso consapevole e considerazione etica.

Cosa vedremo nella seconda parte

Dopo la pausa: Lavoriamo con i Large Language Models "come si usa"

- **Che cos'è un Large Language Model**
- Il **panorama degli strumenti** di IA generativa oggi disponibili
- **ChatGPT** (OpenAI) — il più diffuso
- **Claude** (Anthropic) — l'approccio *safety-first*
- **Gemini** (Google) — multimodalità e integrazione Workspace
- **Microsoft Copilot** — l'IA dentro Office 365
- Un **confronto pratico**: quale strumento per quale compito
- Un **laboratorio guidato**: mettiamo le mani sugli strumenti

Domande?

Pausa

Ci rivediamo tra 15 minuti

Giovanni Della Lunga — giovanni.dellalunga@unibo.it
https://github.com/polyhedron-gdl/Seminari-AI